GESTIÓN DEL PROCESO DE DESARROLLO DE SOFTWARE

Práctica 1

Cazadores de defectos en TiendaUT

Desarrollo, análisis, corrección y evidencia visual. Fecha de ejecución: 15 de octubre de 2026. Fuente de requisitos: guía suministrada, capítulo 19, página 53 y Anexo B.

Objetivo y materiales

Analizar el sitio del ZIP con Playwright, clasificar los fallos, corregir su causa y comprobar estabilidad. Entorno: Windows, Node 24.18.0, Playwright 1.64.1 y axe 4.14.0. Se instalaron dependencias y Chromium, Firefox y WebKit.

Requisito	Cumplimiento comprobado
1. Descomprimir e instalar / publicar	Local realizado. Publicación GitHub pendiente de cuenta.
2. E1-E8 y bitácora	Ocho fallos originales conservados y clasificados.
3. Suite Chromium y seis pasos	12 pruebas aprobadas y 7 fallidas antes de corregir.
4. Corregir el sitio	Carrito, filtro tardío, recursos, accesibilidad y API corregidos.
5. Tres pruebas nuevas	Vaciado, login vacío y contrato de todos los productos.
6. Cinco repeticiones / tres navegadores	315/315 en la entrega inicial; 345/345 en la suite ampliada.
7. Actions y artefacto remoto	Workflow preparado; ejecución remota no realizada.

Este informe documenta la primera de las dos prácticas solicitadas por el estudiante. Para legibilidad, las figuras muestran detalles de capturas reales; los PNG completos se entregan con los reportes HTML, logs, JSON y trazas. Las evidencias locales no se presentan como una ejecución de GitHub.

1. Laboratorio: ocho fallos reproducidos

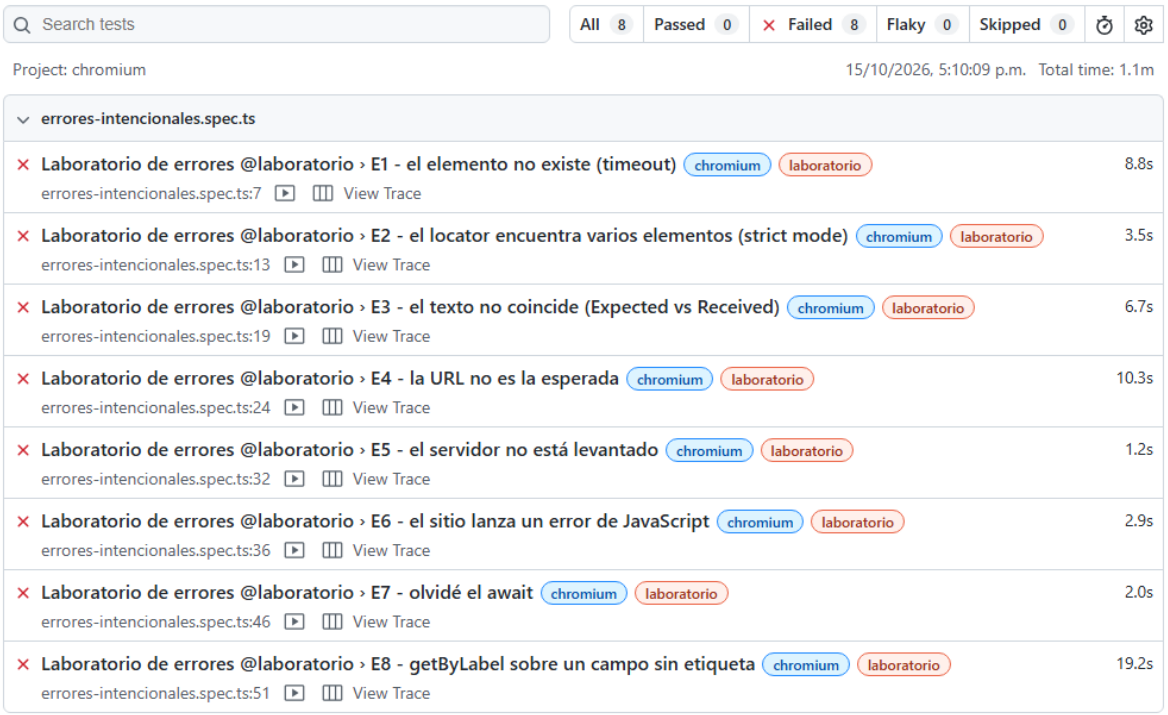


Figura 1. Reporte HTML real del laboratorio original: 8 ejecutadas, 8 fallidas. Los fallos son intencionales; View Trace conserva su evidencia.

ID	Mensaje clave / clasificación	Propuesta de corrección
E1	Timeout: Comprar ahora / prueba	Usar el botón real del producto.
E2	Strict mode: 4 botones / prueba	Acotar por nombre de producto.
E3	Bienvenidos vs. Bienvenido / prueba	Ajustar al encabezado del requisito.
E4	URL sigue en login / prueba	Contraseña inválida: esperar alerta, no redirección.
E5	Connection refused:3999 / entorno	Usar baseURL y servidor en 3000.
E6	reiniciarContador no existe / sitio	Actualizar el contador al vaciar.
E7	Test ended y await ausente / prueba	Esperar la aserción y elegir el encabezado real.
E8	getByLabel sin label / prueba y sitio	Añadir una etiqueta accesible al buscador.

La bitácora extensa y el enlace a cada captura, video y trace.zip están en documentacion/BITACORA.md e INDICE-EVIDENCIAS.md. Los ocho ejemplos se conservan separados de la suite de aceptación.

2. Análisis de los fallos del sitio

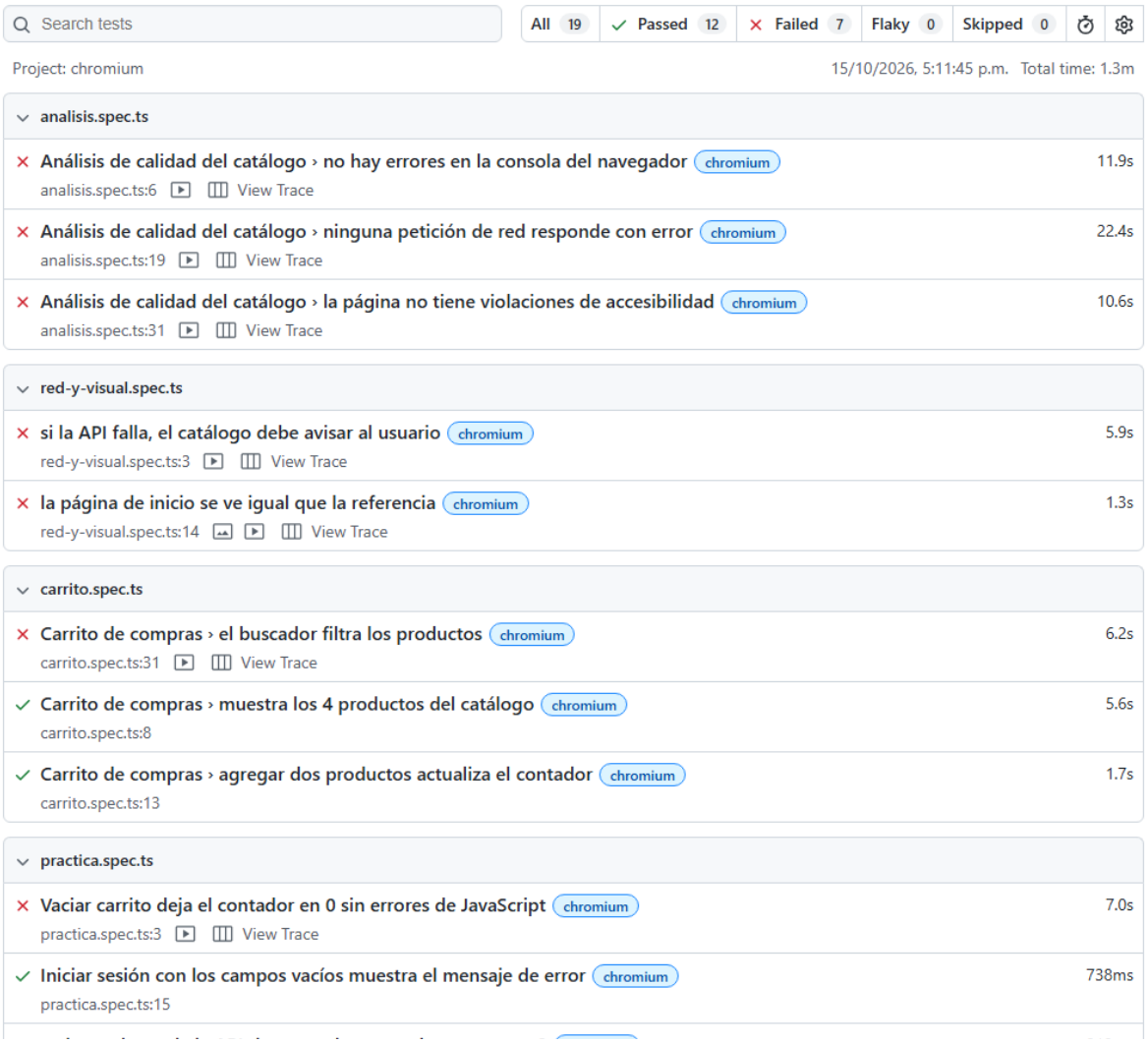


Figura 2. Ejecución original aislada de Chromium, incluidas las tres pruebas nuevas: 12 aprobadas y 7 fallidas.

Método de seis pasos aplicado

1) Leer Expected/Received y el mensaje. 2) Ubicar la acción o aserción. 3) Reproducir con el sitio original. 4) Inspeccionar acciones, DOM y red de la traza. 5) Clasificar como sitio, prueba o entorno. 6) Corregir y volver a comprobar. Las trazas se inspeccionaron programáticamente; se entregan para abrirlas con Trace Viewer.

Causas corregidas

El filtro escrito antes de la API se descartaba; Vaciar carrito llamaba una función inexistente; faltaban logo y favicon; la imagen carecía de alt y el contenido de landmarks; el buscador carecía de etiqueta; un error de API no producía aviso. No se cambiaron las expectativas funcionales originales para ocultar estos defectos.

La falta inicial de una imagen de referencia visual se clasificó como preparación del entorno, no como defecto funcional. También se conservó un intento interferido por el cierre del servidor y dos esperas vencidas en Firefox.

3. Carrito: evidencia antes y después

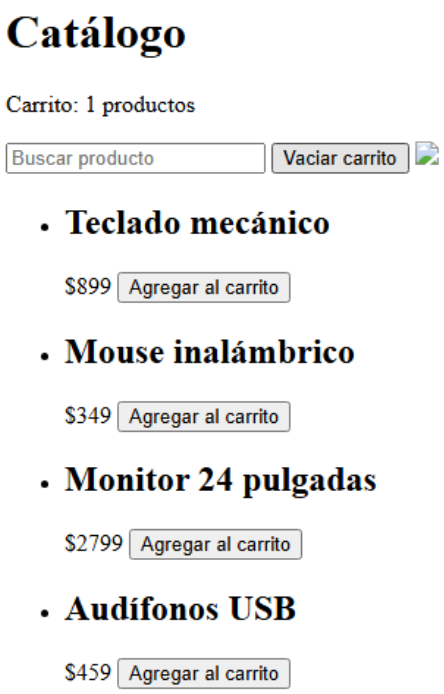


Figura 3. Antes de corregir: después de pulsar Vaciar carrito, el contador permanece en 1. La prueba esperaba 0.



Figura 4. Después de corregir: se agregó un producto, se comprobó 1 y se vació. El contador termina en 0; la prueba también exige cero pageerror.

Causa: reiniciarContador no existía. Corrección: carrito = 0 y contador.textContent = String(carrito). Evidencia: practica.spec.ts, prueba de vaciado; nueva captura reproducida con aserciones en evidencias/capturas/documentacion.spec.ts.

4. Buscador: condición de carrera corregida

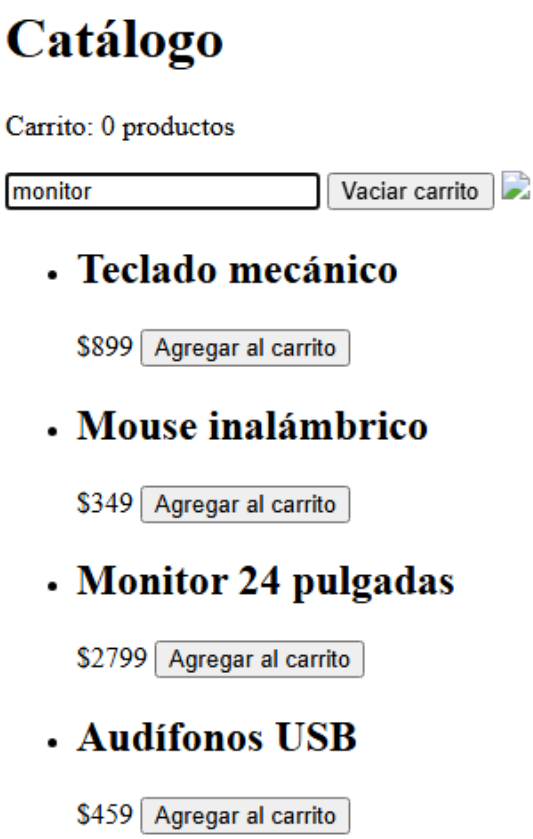


Figura 5. Antes: el buscador contiene monitor, pero aparecen los cuatro productos; la prueba esperaba uno.

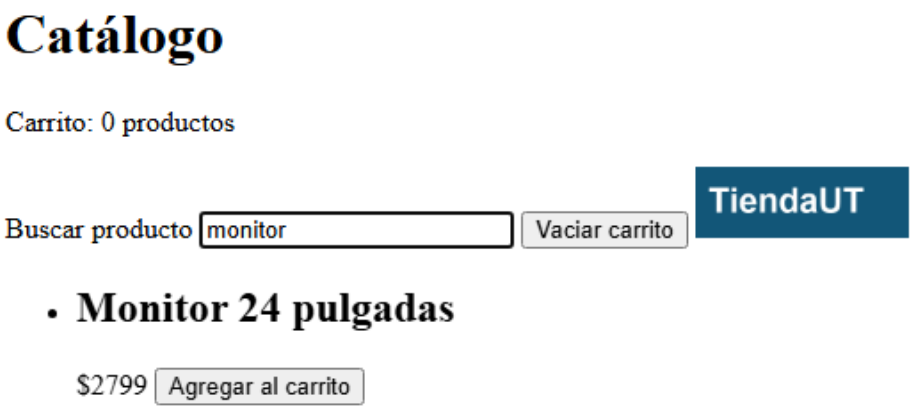


Figura 6. Después: se escribe monitor antes de liberar la respuesta de API y aparece solo Monitor 24 pulgadas.

La traza original registra la escritura antes de terminar la API. Ahora se vuelve a pintar con el valor actual del buscador al recibir los datos. La regresión retiene la petición mediante una promesa y la libera después de escribir; no usa esperas fijas.

5. Las tres pruebas obligatorias añadidas

CP-17: vaciar el carrito sin errores JavaScript

Precondición: carrito con un teclado. Acción: Vaciar carrito. Esperado: contador 0 y lista de pageerror vacía. Su resultado visual está en la figura 4.

CP-18: iniciar sesión con campos vacíos

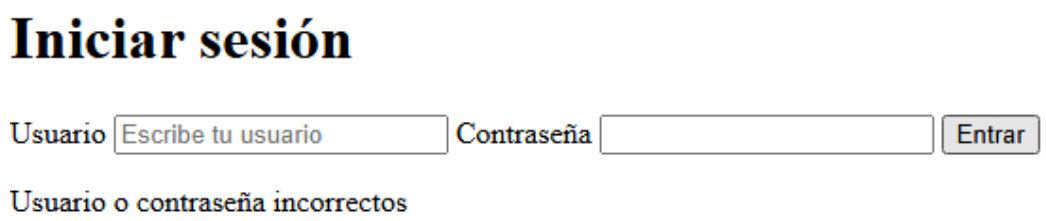


Figura 7. Usuario y contraseña vacíos: aparece Usuario o contraseña incorrectos; la URL sigue siendo login.html.

CP-19: contrato de todos los productos de API

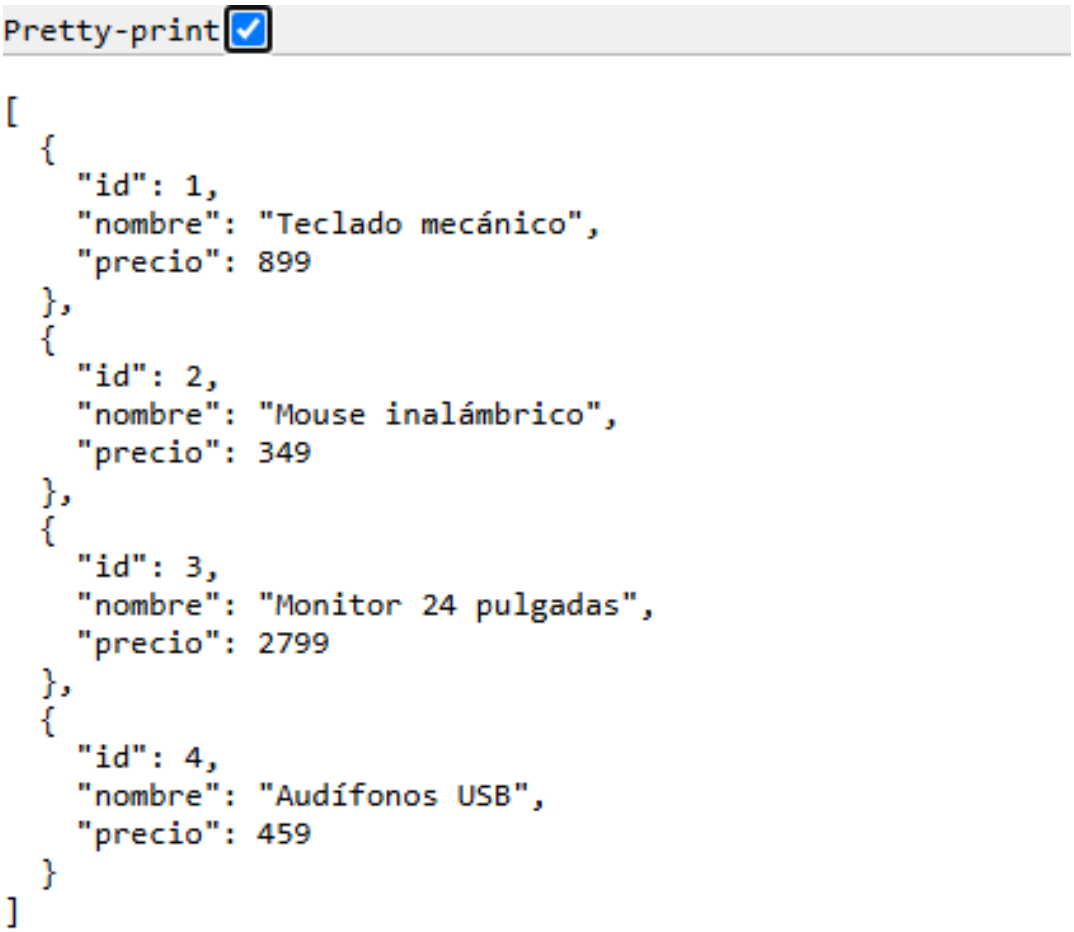


Figura 8. Respuesta real de /api/productos. La prueba recorre todos los productos y exige nombre no vacío y precio numérico finito mayor que cero.

Implementación: tests/practica.spec.ts. Los datos y los resultados comprobados se conservan en el JSON de Playwright; no se infiere el éxito solamente de estas imágenes.

6. Estabilidad y resultado comprobado

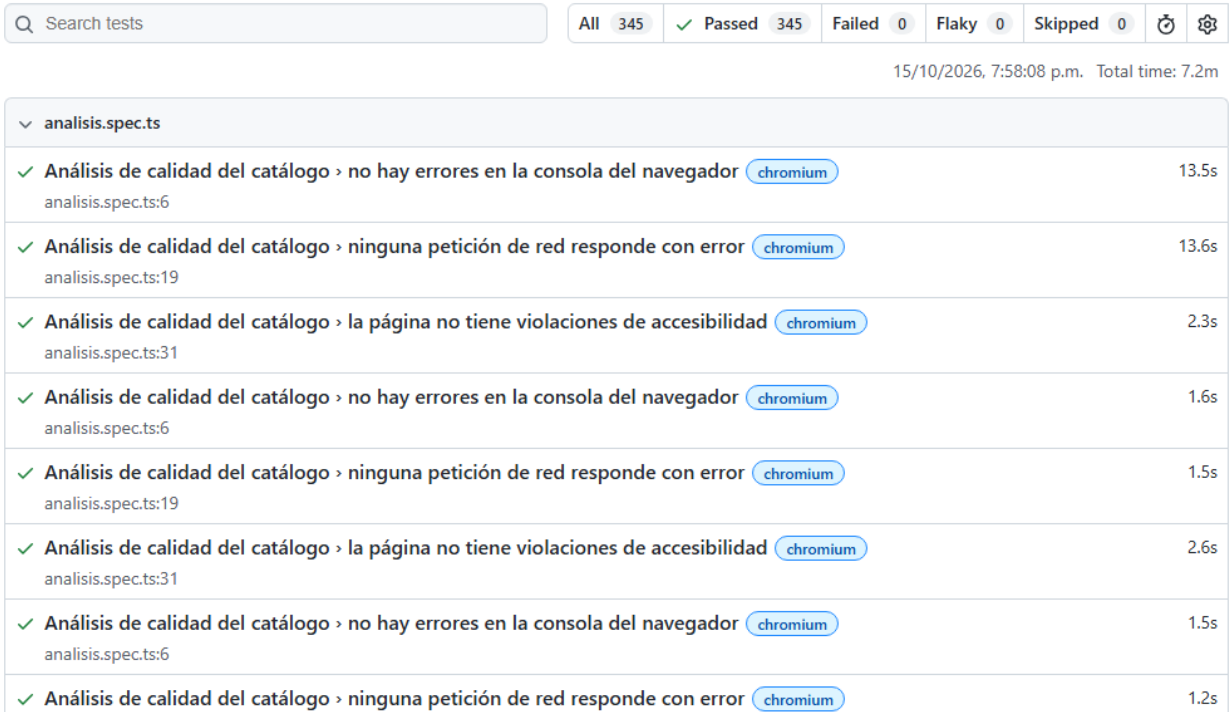


Figura 9. Reporte real de la suite ampliada: 345 aprobadas, cero fallidas, flaky o saltadas. No se actualizaron las imágenes de referencia durante esta comprobación.

Navegador	Casos x repeticiones	Resultado
Chromium	23 x 5	115/115
Firefox	23 x 5	115/115
WebKit	23 x 5	115/115
Total	23 x 5 x 3	345/345

La entrega inicial validó 21 casos (315 ejecuciones); se añadieron dos recorridos completos y se repitió la suite total. No se reemplazó la evidencia histórica. La configuración usa dos workers, 20 segundos para aserciones y 60 por prueba; el margen se justificó tras dos fallos de espera en Firefox y no equivale a una mejora de rendimiento del sitio.

```
npm ci
npx playwright install
npm run lint
npx playwright test --repeat-each=5 --fail-on-flaky-tests
```

7. Red, calidad e integración continua



Figura 10. Un 500 de la API se simula con page.route. El sitio informa No pudimos cargar los productos en una alerta; ya no queda sin explicación.

Calidad comprobada

La captura del catálogo auditado y el adjunto calidad.json acreditan consola sin errores, ninguna respuesta de red >=400 en el camino normal y cero violaciones axe en el catálogo. El caso 500 se valida por separado porque su error de red es intencional.

GitHub Actions: estado pendiente

El workflow instala dependencias y tres navegadores, ejecuta npm run lint y cinco repeticiones y publica HTML, JSON, trazas y referencias visuales durante 30 días incluso al fallar. No se obtuvo una cuenta GitHub: no se creó un repositorio remoto, no se verificó un run en verde ni el artefacto remoto. Estos requisitos siguen pendientes y no tienen una captura simulada.

Para cerrar: conectar una cuenta, crear el repositorio, subir el proyecto, ejecutar Actions y guardar las capturas del run verde y del artefacto descargable. Las instrucciones están en documentacion/GITHUB.md. Las referencias Linux deben revisarse antes de exigir comparación estricta entre cambios.